IoT Security & Cybersecurity

Master 2 — Computer Science

Session 2 of 12 — Cryptography I: Core Encryption

Where We Are

Session 1 closed by observing that nearly every IoT security failure reduces, at some level, to a cryptographic gap. Sessions 2 through 5 close that gap systematically. This session lays the foundation: the formal notion of a cryptosystem, symmetric encryption, asymmetric encryption, and a first, deliberately introductory look at digital signatures. Sessions 3–5 will build directly on top of the vocabulary introduced here.

1 Principles of Cryptography

Definition — Cryptosystem

A cryptosystem is a tuple $(\mathcal{M}, \mathcal{C}, \mathcal{K}, \text{Enc}, \text{Dec})$ where $\mathcal{M}$ is the message space, $\mathcal{C}$ the ciphertext space, $\mathcal{K}$ the key space, and:

$$\text{Enc} : \mathcal{K} \times \mathcal{M} \to \mathcal{C}, \qquad \text{Dec} : \mathcal{K} \times \mathcal{C} \to \mathcal{M},$$

such that for every key $k \in \mathcal{K}$ and message $m \in \mathcal{M}$, $\text{Dec}(k, \text{Enc}(k, m)) = m$ (correctness).

Definition — Kerckhoffs's Principle

A cryptosystem should remain secure even if everything about the system, except the key, is public knowledge. Security must reside in the secrecy of k, never in the secrecy of the algorithm itself.

Example — Kerckhoffs's principle and IoT firmware

An IoT vendor who hides a proprietary, unpublished encryption algorithm inside firmware ("security through obscurity") violates Kerckhoffs's principle. Once the firmware is extracted and reverse-engineered — a routine occurrence for IoT hardware — the entire scheme collapses, because no secret key ever protected it in the first place. Standard, publicly scrutinized algorithms (AES, RSA) remain secure under public knowledge of their design precisely because their security proofs assume the adversary already knows the algorithm.

? Think about it

Give an example that does not respect Kerckhoffs's principle.

✓ Answer

The Mifare Classic RFID card, widely used in transit systems, relied on a proprietary, undisclosed stream cipher (CRYPTO1). Once researchers reverse-engineered it in 2008, they found the cipher was cryptographically weak, and the entire deployed card base — tens of millions of cards — became forgeable, illustrating exactly the failure mode Kerckhoffs's principle warns against.

1.1 Security Goals Revisited

Section 1.3 introduced the CIA triad informally. Cryptography gives these goals precise, adversarial definitions: confidentiality means an adversary observing $\mathrm{Enc}(k, m)$ learns nothing about m beyond its length; integrity means any modification of a ciphertext is detectable; authentication means the receiver can verify the claimed origin of a message. Encryption alone (this session) addresses confidentiality; digital signatures (introduced at the end of this session, extended in Session 3) address integrity and authentication.

2 Symmetric Encryption

Definition — Symmetric Cryptosystem

A cryptosystem is symmetric if encryption and decryption use the same key: $k_{\text{enc}} = k_{\text{dec}} = k$. Both parties must share k over a channel assumed secret before any encrypted communication can begin.

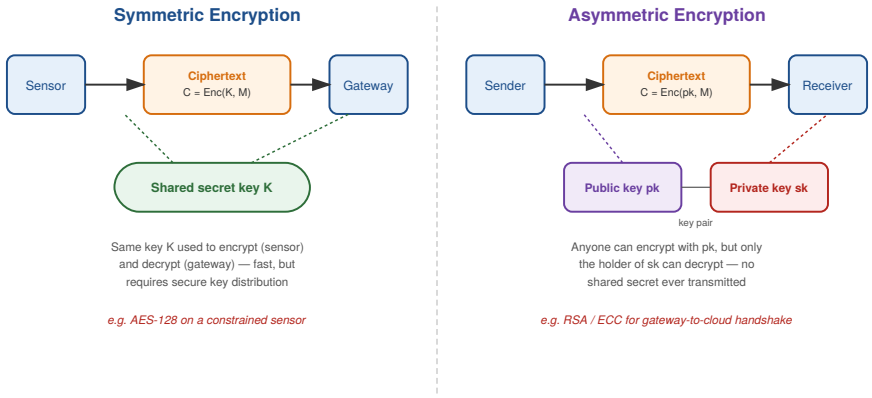


Figure 1: Symmetric encryption (left): sensor and gateway share one secret key. Asymmetric encryption (right): sender encrypts with the receiver's public key; only the matching private key decrypts.

2.1 Block Ciphers and the Substitution–Permutation Network

Most modern symmetric ciphers are block ciphers: they operate on fixed-size blocks of plaintext (e.g., 128 bits) rather than a continuous stream.

Definition — Block Cipher

A block cipher is a family of permutations $E_k : \{0,1\}^n \to \{0,1\}^n$ indexed by a key $k \in \{0,1\}^\kappa$, such that for each fixed k, E_k is a bijection (and hence invertible via $D_k = E_k^{-1}$).

AES (Advanced Encryption Standard), the dominant block cipher today, is built as a substitution–permutation network (SPN): each of several rounds applies a non-linear substitution (an S-box, defeating linear cryptanalysis) followed by a linear permutation (a diffusion layer, spreading the influence of each input bit across the output). AES-128 uses 10 such rounds.

Example — AES on constrained IoT hardware

Full AES-128 requires roughly 2.4 KB of ROM and a few hundred bytes of RAM in a compact software implementation — feasible on most modern microcontrollers, but tight on the smallest 8-bit sensor platforms. This has motivated lightweight cryptography: ciphers like PRESENT (an 80-bit-key SPN cipher with a minimal 4-bit S-box) or SPECK (a simple ARX — add/rotate/xor — cipher designed by the NSA for constrained devices) trade some security margin for a drastically smaller hardware/software footprint.

? Think about it

AES has a fixed block size of 128 bits, but real IoT sensor telemetry (a temperature reading, say) may be only a few bytes long. What problem does this create, and what is the general name for the family of techniques that solves it?

✓ Answer

Encrypting a short message inside a 128-bit block wastes bandwidth (padding overhead is proportionally huge for tiny payloads) and, if done block-by-block naively across multiple messages, can leak patterns. The general solution is a choice of mode of operation (e.g., CTR or GCM mode), which turns a block cipher into a stream-like cipher that can encrypt messages of arbitrary length efficiently, often with an authentication tag included (as in AES-GCM) to protect integrity at the same time.

2.2 Stream Ciphers: The Other Symmetric Family

Not every symmetric cipher operates on fixed blocks. A stream cipher generates a pseudorandom keystream and combines it with plaintext bit-by-bit (typically via XOR), avoiding block-alignment padding entirely.

Definition — Stream Cipher

Given a key k and (optionally) a nonce n, a stream cipher deterministically generates a keystream $Z = z_1 z_2 z_3 \ldots$ of arbitrary length. Encryption is $c_i = m_i \oplus z_i$; decryption is identical, since $m_i \oplus z_i \oplus z_i = m_i$. Security requires that Z be computationally indistinguishable from a truly random bitstream to an adversary without k.

Example — ChaCha20 for constrained devices

ChaCha20, a modern ARX-based stream cipher, is favored in many IoT and mobile contexts over AES specifically because it is fast in pure software (no dedicated hardware acceleration required), unlike AES, which is fast mainly on platforms with AES-NI hardware instructions — rarely present on small microcontrollers. This is why ChaCha20-Poly1305 is a standard alternative cipher suite in TLS 1.3 and is widely used for encrypting IoT traffic on devices lacking AES hardware acceleration.

? Think about it

A stream cipher's security depends critically on never reusing the same (key, nonce) pair to encrypt two different messages. What happens if this rule is violated?

✓ Answer

If the same keystream Z is reused for two messages m_1 and m_2, an attacker who observes $c_1 = m_1 \oplus Z$ and $c_2 = m_2 \oplus Z$ can compute $c_1 \oplus c_2 = m_1 \oplus m_2$ — the keystream cancels out entirely, and the attacker recovers the XOR of the two plaintexts directly, which for structured data (like text) is often enough to recover both messages using frequency analysis, without ever learning the key.

Mode	Mechanism	IoT relevance
ECB	Each block encrypted independently	Insecure: identical plaintext blocks yield identical ciphertext blocks; never used in practice
CBC	Each block XORed with the previous ciphertext before encryption	Requires an unpredictable initialization vector (IV); sequential, harder to parallelize on constrained hardware
CTR	Block cipher used as a keystream generator over a counter	Parallelizable, efficient, but provides no built-in integrity protection
GCM	CTR mode combined with a Galois-field authentication tag	Provides both confidentiality and integrity in one pass; the de facto standard for TLS 1.3 and modern IoT protocols

Example — Why ECB fails visually

Encrypting a bitmap image block-by-block under ECB mode famously still reveals the outline of the original image in the ciphertext, because identical regions of flat color produce identical ciphertext blocks. This is the canonical classroom demonstration of why deterministic, context-free block encryption leaks structural information even while individual blocks remain unreadable.

? Think about it

An IoT vendor cuts corners and encrypts each sensor reading independently under ECB mode to save implementation complexity. An attacker who cannot decrypt anything can still learn something. What?

✓ Answer

Whether two encrypted readings are equal. If a door sensor repeatedly reports "closed" (the same plaintext value), an ECB-encrypted stream will show repeating identical ciphertext blocks whenever the door is closed, letting the attacker infer the door's open/closed pattern over time — a real confidentiality leak — without ever recovering the key or the plaintext value itself.

3 Asymmetric Encryption

Definition — Asymmetric (Public-Key) Cryptosystem

A cryptosystem is asymmetric if it uses a mathematically related but distinct key pair (pk, sk): encryption uses the public key pk, decryption uses the private key sk, and it is computationally infeasible to derive sk from pk alone. Formally, $\text{Enc} : pk \times \mathcal{M} \to \mathcal{C}$ and $\text{Dec} : sk \times \mathcal{C} \to \mathcal{M}$, with $\text{Dec}(sk, \text{Enc}(pk, m)) = m$.

Asymmetric cryptography solves symmetric cryptography's central weakness: key distribution. Two parties who have never met can still communicate confidentially, as long as the receiver's public key is authentically known to the sender — no shared secret ever needs to be transmitted.

3.1 RSA: A Worked Mathematical Example

Definition — RSA Cryptosystem

Key generation: choose two large primes p, q; compute $n = pq$ and Euler's totient $\varphi(n) = (p-1)(q-1)$. Choose public exponent e coprime to $\varphi(n)$, and compute private exponent d such that $ed \equiv 1 \pmod{\varphi(n)}$. The public key is (n, e); the private key is (n, d).

$$\text{Encryption: } c = m^e \bmod n, \qquad \text{Decryption: } m = c^d \bmod n.$$

Correctness follows from Euler's theorem: $m^{ed} \equiv m \pmod{n}$ whenever $\gcd(m, n) = 1$, since $ed \equiv 1 \pmod{\varphi(n)}$.

Example — RSA with small numbers

Let $p = 61$, $q = 53$. Then $n = 3233$ and $\varphi(n) = 60 \times 52 = 3120$. Choose $e = 17$ (coprime to 3120); solving $17d \equiv 1 \pmod{3120}$ gives $d = 2753$. To encrypt $m = 65$: $c = 65^{17} \bmod 3233 = 2790$. To decrypt: $m = 2790^{2753} \bmod 3233 = 65$. (Real RSA uses primes of at least 1024 bits each — this toy example exists purely to make the arithmetic checkable by hand.)

? Think about it

RSA's security rests on the assumed hardness of a specific mathematical problem. Which one, and why does knowing n alone not help an attacker recover d?

✓ Answer

The integer factorization problem: given $n = pq$, finding p and q is believed to be computationally infeasible for sufficiently large primes. Without knowing p and q individually, an attacker cannot compute $\varphi(n) = (p-1)(q-1)$, and without $\varphi(n)$, they cannot compute d from e — the entire security of RSA reduces to the assumed hardness of factoring n.

3.2 Elliptic-Curve Cryptography: Why IoT Prefers It

RSA keys must be large (2048+ bits) to remain secure, which is expensive on constrained IoT hardware. Elliptic-curve cryptography (ECC) achieves comparable security with far smaller keys, because the best known attacks against the elliptic-curve discrete logarithm problem (ECDLP) are fully exponential, whereas the best attacks against RSA's factoring problem are sub-exponential.

Definition — Elliptic-Curve Discrete Logarithm Problem

Given a publicly known base point G on an elliptic curve E and a point $Q = dG$ (point addition performed d times), the ECDLP is: recover the scalar d from G and Q. No efficient (sub-exponential) classical algorithm is known for this problem on well-chosen curves.

Example — Key-size comparison

A 256-bit ECC key (e.g., Curve25519) offers security roughly comparable to a 3072-bit RSA key. For a constrained sensor performing a TLS handshake, this difference — 32 bytes versus 384 bytes per key, and correspondingly faster computation — is often the deciding factor in protocol selection, which is why nearly all modern lightweight IoT security profiles (e.g., DTLS profiles for constrained devices) mandate ECC over RSA.

? Think about it

What is the equation of ECC? Give me an example that uses ECC.

✓ Answer

Over a Prime Field: $y^2 = x^3+ax+b \mod p$, where $4a^3+27b^2 \neq 0 \mod p$. Over a Binary Field (For hardware-optimized implementations): $y^2 + xy = x^3 + ax^2 + b$, where $b \neq 0$. Bluetooth Low Energy's Secure Connections pairing mode uses ECDH (elliptic-curve Diffie–Hellman) specifically because BLE chips are designed for coin-cell battery life; an RSA-based handshake would consume a disproportionate amount of the device's energy budget for a single pairing operation.

3.3 Hybrid Encryption: Combining Both Worlds

In practice, almost no real system uses asymmetric encryption alone to protect bulk data. Asymmetric operations (modular exponentiation, elliptic-curve point multiplication) are orders of magnitude slower than symmetric block-cipher operations, especially on constrained hardware. Hybrid encryption resolves this by using each primitive for what it is best at.

Definition — Hybrid Encryption

To encrypt a message m for a receiver with public key pk: (1) generate a fresh random symmetric key k (a session key); (2) encrypt the bulk message with the fast symmetric cipher, $c_1 = \mathrm{Enc}_{\mathrm{sym}}(k, m)$; (3) encrypt only the short session key with the slow asymmetric cipher, $c_2 = \mathrm{Enc}_{\mathrm{asym}}(pk, k)$; (4) transmit (c_1, c_2). The receiver reverses the order: recover k from c_2 using sk, then recover m from c_1 using k.

Example — TLS as hybrid encryption

Every HTTPS connection (and its IoT counterpart, DTLS) is a hybrid scheme in this exact sense: the TLS handshake uses asymmetric operations (certificate-based key exchange) only to establish a shared symmetric session key, after which all bulk application data is encrypted with a fast symmetric cipher (typically AES-GCM or ChaCha20-Poly1305). This is precisely why TLS handshakes are the "expensive" part of a connection and bulk data transfer afterward is cheap.

? Think about it

A constrained sensor needs to send one 20-byte reading per minute to the cloud. Why would establishing a brand-new hybrid TLS session for every single 20-byte reading be wasteful, and

what does this motivate?

✓ Answer

The asymmetric handshake cost would dwarf the cost of encrypting the tiny payload itself — the overhead is almost entirely in session establishment, not bulk encryption. This motivates session resumption mechanisms (e.g., TLS session tickets or DTLS connection IDs), which let a device reuse a previously negotiated symmetric session key across many short messages instead of repeating the full asymmetric handshake each time.

3.4 Padding and a Cautionary Tale

Asymmetric and block-cipher schemes typically require the input to be padded to a specific format before the mathematical operation is applied. Getting padding wrong has historically been a major source of practical breaks.

Example — Padding oracle attacks

If a system decrypts a ciphertext and reveals — even indirectly, e.g. through a timing difference or a distinct error message — whether the padding was well-formed, an attacker can exploit this single bit of leaked information across many queries to decrypt the ciphertext entirely, without ever learning the key. This class of attack (padding oracle attacks) is why modern designs strongly prefer padding schemes and cipher modes with built-in integrity checking (such as OAEP for RSA, or AES-GCM instead of unauthenticated CBC), so that a tampered or malformed ciphertext is rejected outright rather than decrypted and inspected.

? Think about it

Give an example from the real world.

✓ Answer

The 2014 "POODLE" attack exploited exactly this padding-oracle weakness in SSL 3.0's CBC-mode padding, allowing attackers to decrypt small portions of encrypted web traffic byte-by-byte — a major motivator for the industry-wide move away from CBC-based cipher suites toward AEAD (authenticated encryption with associated data) modes like GCM.

Encryption protects confidentiality. It does not, by itself, prove who sent a message or that the message was not altered in transit. Digital signatures close this gap, and this section opens the topic that Session 3 will extend substantially (ring, group, threshold, and other advanced signature variants).

Definition — Digital Signature Scheme

A digital signature scheme is a triple of algorithms $(\text{KeyGen}, \text{Sign}, \text{Verify})$:

$$\text{KeyGen} \to (pk, sk), \qquad \text{Sign}(sk, m) \to \sigma, \qquad \text{Verify}(pk, m, \sigma) \to \{0, 1\},$$

such that $\text{Verify}(pk, m, \text{Sign}(sk, m)) = 1$ for every valid key pair and message (correctness), and forging a valid σ for a new message without knowledge of sk is computationally infeasible (unforgeability).

Crucially, signature schemes sign a hash of the message rather than the message itself, both for efficiency (fixed-size input regardless of message length) and because some signature algorithms (like RSA's textbook form) are only secure when applied to a value with specific mathematical properties that a good hash function provides.

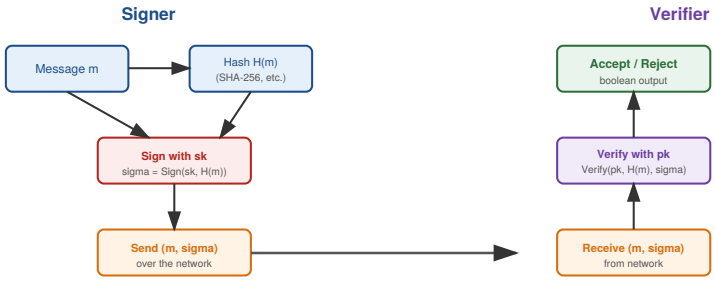


Figure 2: The sign/verify workflow. The signer hashes the message, signs the hash with their private key, and sends both; the verifier recomputes the hash and checks it against the signature using the signer's public key.

3.5 RSA Signatures: Reusing the Same Trapdoor

Definition — RSA Signature (simplified)

Using the same key pair (n, e, d) as RSA encryption, but with roles reversed:

$$\text{Sign: } \sigma = H(m)^d \bmod n, \qquad \text{Verify: accept iff } \sigma^e \bmod n = H(m).$$

Only the holder of d can produce a valid σ; anyone with the public (n, e) can verify it.

Example — Signed firmware updates

A device manufacturer signs each firmware image with their private key before publication. Every IoT device ships with the manufacturer's public key baked into ROM at manufacturing time. Before installing any update, the device verifies the signature; an attacker who intercepts the update channel and substitutes malicious firmware cannot produce a valid signature without the manufacturer's private key, so the device rejects the forged update.

? Think about it

Suppose a device's secure-boot process checks the firmware signature but the manufacturer's public key itself is stored in writable, unprotected flash memory. What attack does this open, even though the signature verification logic is otherwise correct?

✓ Answer

An attacker with physical or remote write access to flash can simply replace the stored public key with their own, then sign malicious firmware with the matching private key they control — the device's verification logic will report success because it is checking against the wrong key. This illustrates why the public key (or a hash of it) must be stored in immutable, write-protected memory (e.g., one-time-programmable fuses), not merely alongside the firmware it is meant to authenticate.

3.6 A First Look Beyond Single Signatures

Textbook RSA/ECDSA signatures above assume one signer and one verifier. Real IoT and distributed systems frequently need richer guarantees: several parties jointly authorizing an action, or the signer's identity itself needing to stay hidden. A multi-signature scheme, at its simplest, lets n signers jointly produce a single compact signature σ over a message m, such that verification

confirms all n parties participated, without σ growing linearly with n.

Example — Why IoT needs multi-party authorization

Consider a smart-lock system requiring both a homeowner's device and a monitoring service to jointly authorize a remote unlock command, so that neither party alone can unlock the door. A multi-signature scheme allows both signatures to be verified together as a single compact proof, rather than the door controller needing to store and check two full, independent signatures.

? Think about it

This session ends on a deliberately incomplete definition of multi-signatures. What specific question about "jointly producing a single compact signature" do you expect Session 3 to have to answer mathematically?

✓ Answer

How the individual signers' partial signatures are combined into one compact σ without any single signer having to reveal their private key to the others, and how verification can confirm that all required signers (not just one) actually participated — this combination mechanism, along with related constructions such as ring, threshold, and blind signatures, is exactly what Session 3 formalizes.

Session Summary

This session established the cryptosystem formalism, walked through symmetric encryption (block ciphers, AES's SPN structure, and modes of operation) and asymmetric encryption (RSA with a fully worked numeric example, and ECC's efficiency advantage for constrained IoT hardware), and opened the topic of digital signatures with the single-signer RSA construction. Session 3 extends signatures into their full advanced family — ring, aggregated, group, blind, and threshold signatures — together with key exchange protocols and certification.